

¿Qué son los estados en React?

Un estado en React es un objeto interno que cada componente de React mantiene para almacenar datos que pueden cambiar a lo largo del tiempo. Cuando el estado cambia, React renderiza de nuevo el componente afectado para reflejar esos cambios en la interfaz, actualizando solo lo necesario en el DOM real.

¿Cuáles son sus características y para qué sirve?

Dentro de sus principales características encontramos:

- **Puede ser asincrónico**, ya que React puede agrupar varias actualizaciones de estado por razones de performance.
- **Re-renderiza**, ya que cada actualización de estado provoca que React vuelva a renderizar el componente.
- **Mutabilidad controlada**, ya que solo se modifica mediante funciones específicas como `setState`; nunca de forma directa.

El estado en React permite construir interfaces dinámicas e interactivas, desde mostrar u ocultar elementos, llevar el valor de un formulario, contar clicks, etc. Es el mecanismo que tiene React para darle “memoria” a un componente.

Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descripto, explicando el ejemplo.

<https://github.com/GabrielLeiva05/IS2-2026-LEIVA/blob/master/EJ4/i.Investigacion-React/EJEMPLOS/ejemplos-react/src/Ejemplo1.jsx>

En `Ejemplo1.jsx` ejemplificamos el uso de estados de React con un contador. `useState(0)` devuelve un arreglo con el valor del contador y una función para actualizarlo. Cada vez que se presione el botón de incrementar, decrementar o reiniciar el valor del contador, React detecta ese cambio, vuelve a renderizar el componente y actualiza el valor según corresponda.

¿Qué son los eventos en React?

Los eventos en React son la forma en la que los componentes reaccionan a las acciones del usuario, ya sean clicks, escritura en algún campo, envío de formulario, etc; o también acciones del sistema. React implementa un sistema propio llamado SyntheticEvent, que es un wrapper sobre los eventos nativos del navegador que unifica su comportamiento entre distintos navegadores.

¿Cuáles son sus características y para qué sirve?

Entre sus principales características encontramos:

- **Se escriben en camelCase**, por ejemplo, onClick, onChange, onSubmit, etc.
- **Al atributo del evento se le asigna una referencia a una función.**
- React no agrega un listener por cada elemento del DOM; internamente delega la escucha de forma centralizada, mejorando el rendimiento.

Los eventos permiten capturar la interacción del usuario con la interfaz y ejecutar lógica en respuesta, como por ejemplo, validar un formulario, actualizar datos en tiempo real, mostrar mensajes, etc.

Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descripto, explicando el ejemplo.

<https://github.com/GabrielLeiva05/IS2-2026-LEIVA/blob/master/EJ4/i.Investigacion-React/EJEMPLOS/ejemplos-react/src/Ejemplo2.jsx>

Ejemplo 2 representa un formulario que al presionar un botón saluda con el nombre que se ingresa. Cada tecla del nombre que vamos ingresando dispara el evento onChange. Al enviar el formulario se dispara onSubmit y se actualiza el mensaje, lo que resulta en un nuevo renderizado mostrando el saludo.

¿Qué son los Hook en React?

Los Hooks en React son funciones especiales que permiten a los componentes funcionales usar características que antes solo estaban disponibles en los componentes de clase, como el manejo de estado, sin necesidad de escribir una clase.

¿Cuáles son sus características y para qué sirve?

Dentro de sus principales características podemos encontrar:

- **Nombre con prefijo 'use'**: Todo Hook comienza con la palabra 'use', como useState, useEffect.
- Deben llamarse siempre en el **cuerpo principal del componente o dentro de otro Hook**, nunca dentro de condicionales, bucles o funciones anidadas.

Los Hooks permiten escribir componentes funcionales completos, es decir, con estado propio, efectos secundarios como llamadas a APIs, acceso al contexto global, etc. Simplifican código y facilitan la reutilización lógica entre componentes.

¿Cuáles son los principales Hook?

Dentro de los principales Hook en React, nos encontramos los siguientes:

- **useState:** declara una variable de estado y su función para actualizarla.
- **useEffect:** ejecuta efectos secundarios, tales como fetch de datos, timers, etc; después del renderizado y permite limpiarlos.
- **useContext:** permite consumir un contexto (datos globales) sin necesidad de pasar props manualmente por cada nivel.
- **useMemo/ useCallback :** memorizan valores o funciones para evitar cálculos innecesarios, optimizando el rendimiento.

Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descripto, explicando el ejemplo.

<https://github.com/GabrielLeiva05/IS2-2026-LEIVA/blob/master/EJ4/i.Investigacion-React/EJEMPLOS/ejemplos-react/src/Ejemplo3.jsx>

Ejemplo3 representa un reloj digital en el cual `useState(new Date())` guarda la hora actual en hora. `useEffect` define un efecto secundario que crea un `setInterval` para actualizar la hora cada 1s. Al cambiar, React vuelve a renderizar el componente y muestra la nueva hora. `[]` representa que este efecto que mencionamos debe ejecutarse una sola vez, cuando se monta el componente. La función que retorna es la función de limpieza, la cual React la ejecuta automáticamente cuando el componente se desmonta, evitando memory leaks.